

1 Problem Definition

In this assignment, different sorting algorithms are implemented and compared based on their performance. The algorithms used are Quick Sort, Insertion Sort, Merge Sort, Shell Sort, and Radix Sort.

The goal is to see how these algorithms behave with different input sizes and different types of data such as random, sorted, and reverse sorted arrays. Their running times are measured and compared to understand their efficiency.

2 Solution Implementation

Five sorting algorithms were implemented in Java following the given pseudocode. The volume column from the dataset was extracted and used as input.

For each input size, three datasets were prepared: random, sorted, and reverse sorted. Each algorithm was executed 10 times on each dataset, and the average execution time was recorded in milliseconds using `System.nanoTime()`.

Only the sorting process was measured, and each run was performed on a copy of the data to ensure fairness. The results were presented in tables and graphs for comparison.

2.1 Quick Sort

Quick Sort is a divide-and-conquer sorting algorithm. It works by selecting a pivot element and partitioning the array into two parts: elements smaller than the pivot and elements greater than the pivot. Then, the same process is applied to the subarrays.

In this implementation, the last element is chosen as the pivot, and an iterative version of Quick Sort is used with an explicit stack instead of recursion.

```
1 public class QuickSort {
2
3     public static void quickSort(int[] A, int low, int high) {
4
5         int stackSize = high - low + 1;
6         int[] stack = new int[stackSize];
7
8         int top = -1;
9
10        top = top + 1;
11        stack[top] = low;
12
13        top = top + 1;
14        stack[top] = high;
15
16        while (top >= 0) {
17
18            high = stack[top];
```

```

19         top = top - 1;
20
21         low = stack[top];
22         top = top - 1;
23
24         int pivot = partition(A, low, high);
25
26         if (pivot - 1 > low) {
27             top = top + 1;
28             stack[top] = low;
29
30             top = top + 1;
31             stack[top] = pivot - 1;
32         }
33
34         if (pivot + 1 < high) {
35             top = top + 1;
36             stack[top] = pivot + 1;
37             top = top + 1;
38             stack[top] = high;
39         }
40     }
41 }
42
43 public static int partition(int[] A, int low, int high) {
44
45     int pivot = A[high];
46     int i = low - 1;
47
48     for (int j = low; j <= high - 1; j++) {
49
50         if (A[j] <= pivot) {
51             i = i + 1;
52
53             int temp = A[i];
54             A[i] = A[j];
55             A[j] = temp;
56         }
57     }
58
59     int temp = A[i + 1];
60     A[i + 1] = A[high];
61     A[high] = temp;
62
63     return i + 1;
64 }
65 }

```

2.2 Insertion Sort

Insertion Sort is a simple sorting algorithm that builds the sorted array one element at a time. At each step, it takes the next element and places it into its correct position among the already sorted elements.

This algorithm is efficient for small datasets and performs very well when the input is already sorted.

```
66 public class InsertionSort {
67
68     public static void insertionSort(int[] A) {
69         for (int j = 1; j < A.length; j++) {
70             int key = A[j];
71             int i = j - 1;
72             while (i >= 0 && A[i] > key) {
73                 A[i + 1] = A[i];
74                 i = i - 1;
75             }
76             A[i + 1] = key;
77         }
78     }
79 }
```

2.3 Merge Sort

Merge Sort is a divide-and-conquer algorithm that splits the array into smaller parts, sorts them, and then merges them back together in order.

In this implementation, an iterative (bottom-up) version is used instead of recursion. Subarrays are merged step by step by increasing their size until the whole array is sorted.

```
80 public class MergeSort {
81
82     public static void mergeSort(int[] A) {
83         int n = A.length;
84         int[] temp = new int[n];
85         int currSize = 1;
86         while (currSize < n) {
87             int leftStart = 0;
88             while (leftStart < n - 1) {
89                 int mid = Math.min(leftStart + currSize - 1, n - 1);
90                 int rightEnd = Math.min(leftStart + 2 * currSize - 1, n - 1);
91                 merge(A, temp, leftStart, mid, rightEnd);
92                 leftStart = leftStart + 2 * currSize;
93             }
94             currSize = 2 * currSize;
95         }
96     }
```

```

97
98     public static void merge(int[] A, int[] temp, int left, int mid, int right
99     ) {
100         int i = left;
101         int j = mid + 1;
102         int k = left;
103         while (i <= mid && j <= right) {
104             if (A[i] <= A[j]) {
105                 temp[k] = A[i];
106                 i = i + 1;
107             } else {
108                 temp[k] = A[j];
109                 j = j + 1;
110             }
111             k = k + 1;
112         }
113         while (i <= mid) {
114             temp[k] = A[i];
115             i = i + 1;
116             k = k + 1;
117         }
118         while (j <= right) {
119             temp[k] = A[j];
120             j = j + 1;
121             k = k + 1;
122         }
123         for (i = left; i <= right; i++) {
124             A[i] = temp[i];
125         }
126     }

```

2.4 Shell Sort

Shell Sort is an improved version of Insertion Sort. Instead of comparing adjacent elements, it compares elements that are far apart using a gap value. The gap is gradually reduced until it becomes 1, and then a final insertion sort step is performed.

In this implementation, the Knuth gap sequence is used to determine the gap values.

```

127 public class ShellSort {
128     public static void shellSort(int[] A) {
129         int n = A.length;
130         int h = 1;
131         while (h < n/3) {
132             h = 3*h + 1;
133         }
134         while (h >= 1) {

```

```

135         for(int i = h; i < n; i++) {
136             int j = i;
137             while(j >= h && A[j] < A[j-h]) {
138                 int temp = A[j];
139                 A[j] = A[j-h];
140                 A[j-h] = temp;
141                 j = j - h;
142             }
143         }
144         h = h/3;
145     }
146 }
147 }

```

2.5 Radix Sort

Radix Sort is a non-comparison based sorting algorithm. It sorts the elements digit by digit, starting from the least significant digit to the most significant digit.

In this implementation, counting sort is used as a subroutine to sort the elements based on each digit. This process is repeated for all digit positions.

```

148 public class RadixSort {
149
150     public static int[] radixSort(int[] A, int d) {
151         for (int pos = 1; pos <= d; pos++) {
152             A = countingSort(A, pos);
153         }
154         return A;
155     }
156
157     public static int[] countingSort(int[] A, int pos) {
158         int[] count = new int[10];
159         int[] output = new int[A.length];
160         int size = A.length;
161
162         for (int i = 0; i < size; i++) {
163             int digit = getDigit(A[i], pos);
164             count[digit]++;
165         }
166
167         for (int i = 1; i < 10; i++) {
168             count[i] += count[i - 1];
169         }
170
171         for (int i = size - 1; i >= 0; i--) {
172             int digit = getDigit(A[i], pos);
173             count[digit]--;

```

```

174         output[count[digit]] = A[i];
175     }
176
177     return output;
178 }
179
180 public static int getDigit(int number, int pos) {
181     return (number / (int) Math.pow(10, pos - 1)) % 10;
182 }
183 }

```

3 Results, Analysis, Discussion

Table 1: Execution times of sorting algorithms (in ms)

Algorithm	500	1000	2000	4000	8000	16000	32000	64000	128000	250000
Random Input Data Timing Results (ms)										
Quick Sort	0.11	0.07	0.20	0.21	0.43	0.78	1.60	10.85	14.40	58.26
Insertion Sort	0.40	0.26	0.66	4.97	16.03	64.89	413.68	2863.08	5933.90	35499.18
Merge Sort	0.15	0.14	0.32	0.29	0.43	0.94	4.43	13.84	22.31	59.52
Shell Sort	0.18	0.12	0.16	0.27	0.46	1.05	5.01	16.79	26.73	87.50
Radix Sort	0.77	0.77	0.11	0.13	0.18	0.35	1.28	4.99	6.48	20.08
Sorted Input Data Timing Results (ms)										
Quick Sort	0.54	0.54	2.31	9.90	34.23	147.95	1075.37	4598.82	17946.83	95042.70
Insertion Sort	0.18	0.00	0.00	0.01	0.01	0.01	0.05	0.07	0.26	0.33
Merge Sort	0.23	0.09	0.06	0.16	0.14	0.25	1.26	2.23	8.27	10.28
Shell Sort	0.06	0.00	0.01	0.06	0.04	0.07	0.38	0.74	2.41	3.33
Radix Sort	0.47	0.70	0.07	0.14	0.21	0.35	1.44	3.03	10.05	10.81
Reverse Sorted Input Data Timing Results (ms)										
Quick Sort	0.24	0.48	1.65	7.38	19.39	40.82	152.94	362.54	1806.92	3410.40
Insertion Sort	0.24	0.50	2.21	7.78	24.34	93.21	1290.40	2977.54	19344.55	51158.61
Merge Sort	0.25	0.09	0.05	0.08	0.16	0.34	2.47	3.30	10.59	47.81
Shell Sort	0.02	0.01	0.02	0.04	0.09	0.17	1.23	1.44	5.26	17.67
Radix Sort	0.34	0.04	0.06	0.09	0.19	0.42	2.45	5.05	10.73	35.93

Table 2: Computational complexity comparison of the given algorithms.

Algorithm	Best Case	Average Case	Worst Case
Quick Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$
Insertion Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$
Shell Sort	$\Omega(n \log n)$	$\Theta(n^{3/2})$	$O(n^2)$
Radix Sort	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$

Table 3: Auxiliary space complexity of the given algorithms.

Algorithm	Auxiliary Space Complexity
Quick sort	$O(n \log n)$
Insertion sort	$O(1)$
Merge sort	$O(n)$
Shell sort	$O(1)$
Radix sort	$O(n + k)$

The observed results are consistent with the theoretical complexities of the algorithms.

- **Quick Sort:** In general, Quick Sort runs in $O(n \log n)$ time when the pivot divides the array into balanced parts. However, if a poor pivot is selected (for example, choosing the last element in an already sorted array), the performance degrades to $O(n^2)$. Since an iterative version is used, the auxiliary space is $O(\log n)$ due to the stack.
- **Insertion Sort:** In the best case, when the array is already sorted, the algorithm completes in $\Omega(n)$ time because no shifting is needed. In average and worst cases, elements must be shifted repeatedly, resulting in $O(n^2)$ time. It works in-place, so it requires $O(1)$ extra space.
- **Merge Sort:** Merge Sort always divides the array into smaller parts and merges them, giving a consistent $O(n \log n)$ running time for all cases. However, it requires an additional temporary array during merging, which leads to $O(n)$ auxiliary space.
- **Shell Sort:** The time complexity of Shell Sort depends on the gap sequence used. With the Knuth sequence, the average case is around $O(n^{1.5})$, while the worst case can reach $O(n^2)$. It is an in-place algorithm, so it uses $O(1)$ extra space.
- **Radix Sort:** Radix Sort processes the input digit by digit and performs k passes over n elements. Therefore, its time complexity is $\Omega(nk)$ in the best case, $\Theta(nk)$ on average, and $O(nk)$ in the worst case. It uses additional arrays for counting and output, so its auxiliary space is $O(n + k)$.

Random Input Comparison

This graph shows the running times of all sorting algorithms on random input data. As the input size increases, the difference between the algorithms becomes more visible. Efficient algorithms like Quick Sort, Merge Sort, and Radix Sort perform significantly better than Insertion Sort for large inputs.

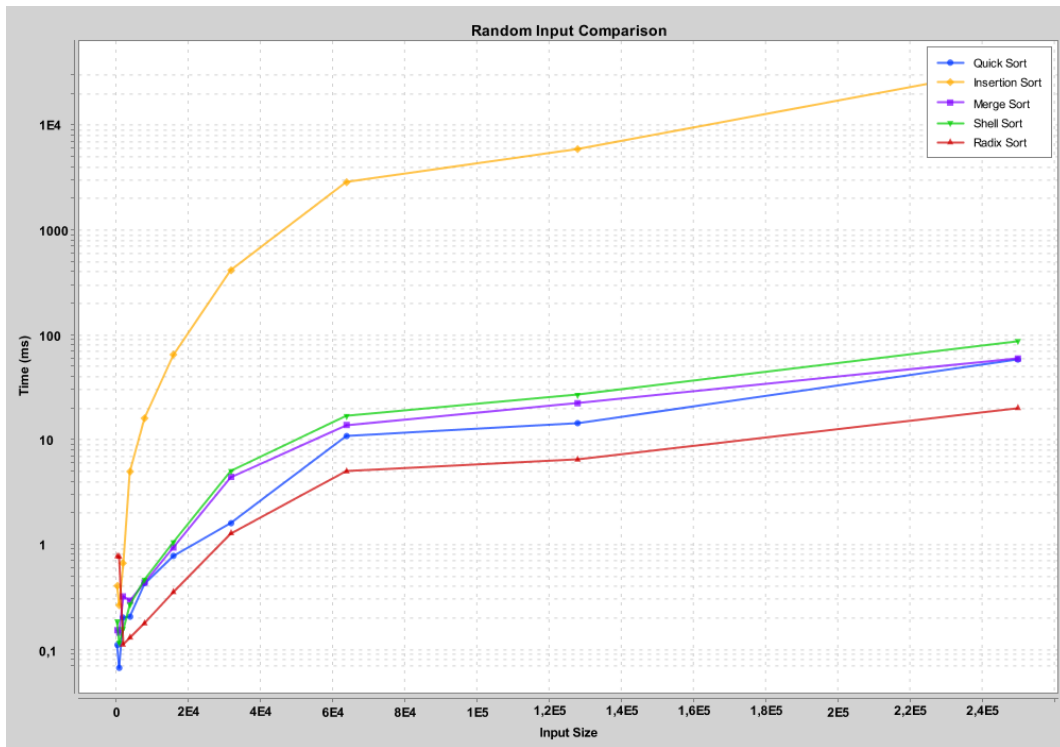


Figure 1: Running times of all sorting algorithms on random input data.

Sorted Input Comparison

This graph presents the performance of the algorithms on already sorted data. In this case, Insertion Sort performs very efficiently, while Quick Sort performs poorly due to bad pivot selection. Merge Sort and Radix Sort maintain stable performance regardless of input order.

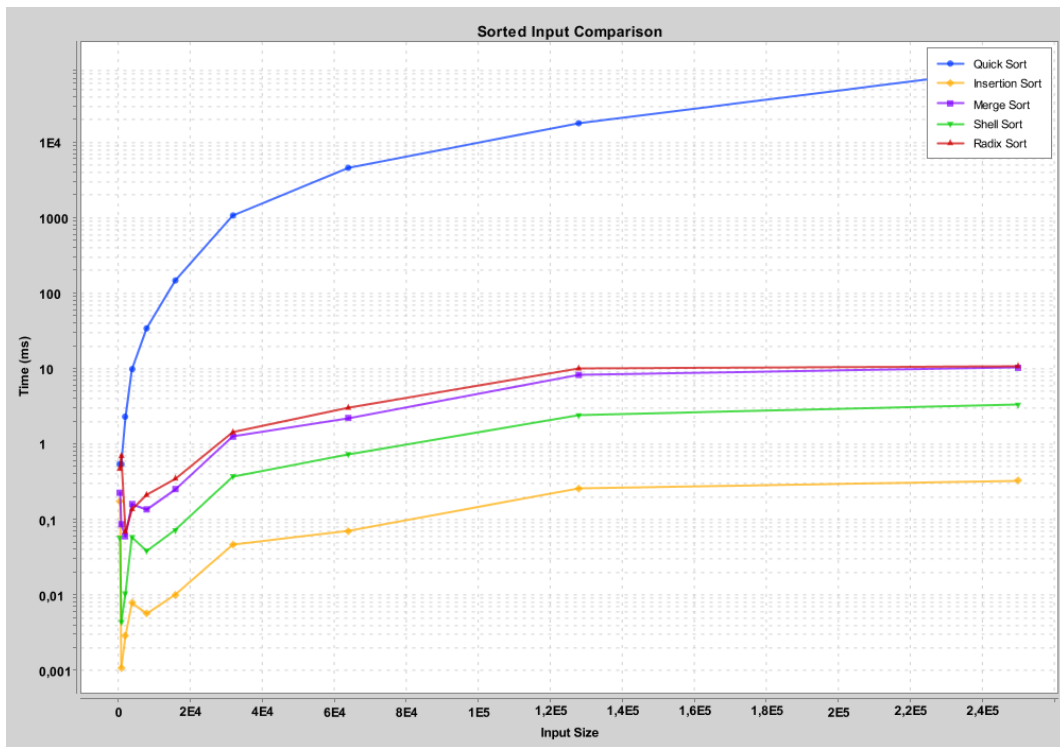


Figure 2: Running times of all sorting algorithms on sorted input data.

Reverse Sorted Input Comparison

This graph shows the running times on reverse sorted input data. Insertion Sort performs very poorly in this case due to its quadratic complexity. Quick Sort also shows performance degradation, while Merge Sort, Shell Sort, and Radix Sort remain relatively stable.

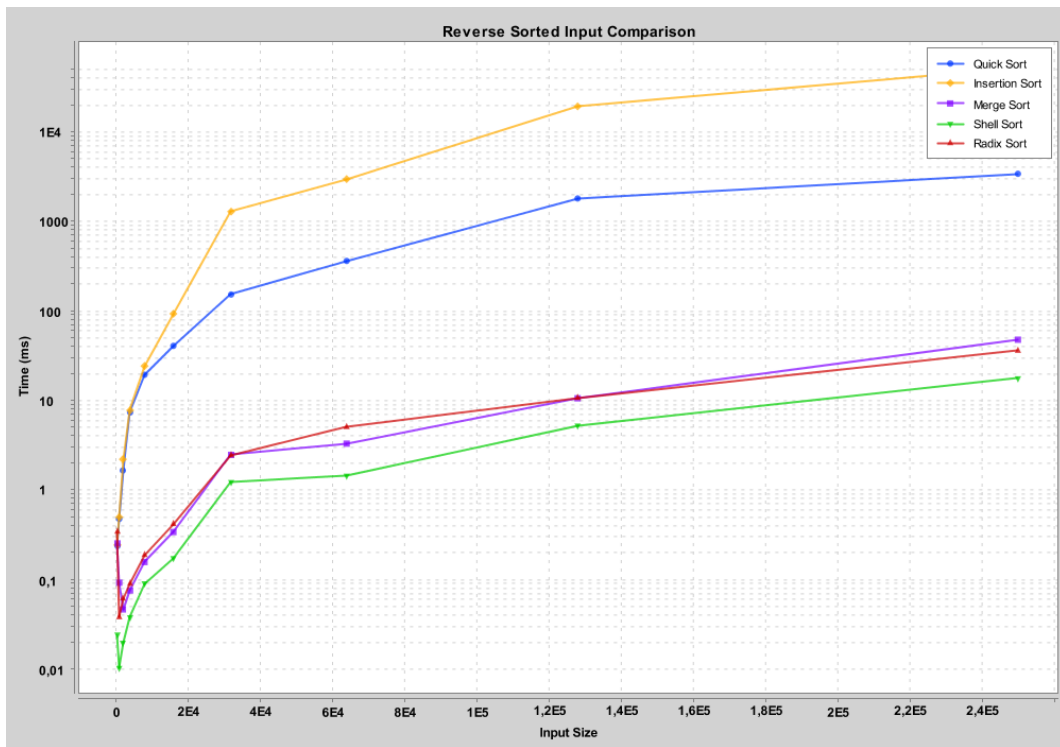


Figure 3: Running times of all sorting algorithms on reverse sorted input data.

Quick Sort Performance Analysis

This graph focuses on Quick Sort and compares its performance under different input orderings. It performs well on random input but significantly worse on sorted and reverse sorted inputs. This behavior clearly matches the theoretical worst-case complexity of Quick Sort.

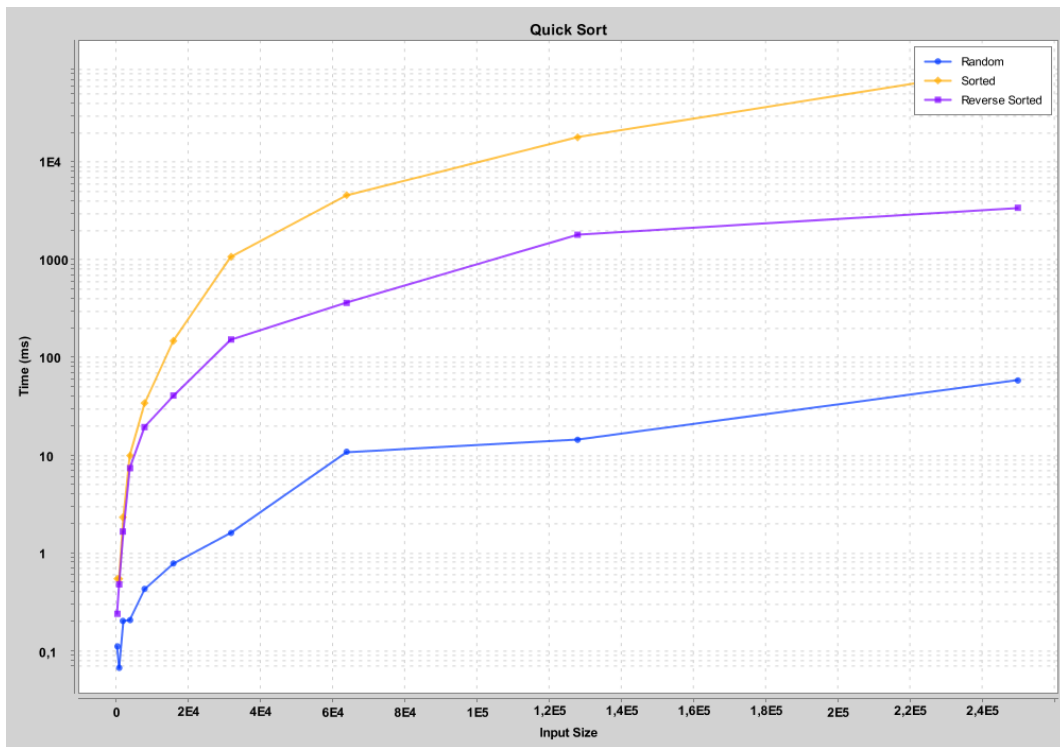


Figure 4: Performance of Quick Sort under different input orderings.

4 Results Analysis and Discussion

Q1: What are the best-, average-, and worst-case time complexities of the given algorithms? For each case, describe the input characteristics that trigger these behaviors.

Answer:

- **Quick Sort:** Best and average case time complexity is $O(n \log n)$ when the pivot divides the array into roughly equal parts. This usually happens with random input. The worst case is $O(n^2)$, which occurs when the pivot choice is poor, such as always picking the last element in an already sorted or reverse sorted array.
- **Insertion Sort:** The best case is $\Omega(n)$ when the array is already sorted, since no shifting is needed. The average and worst cases are $O(n^2)$, which happen when the array is random or reverse sorted, requiring many comparisons and shifts.
- **Merge Sort:** The time complexity is $O(n \log n)$ in best, average, and worst cases. This is because the algorithm always divides the array and merges it in the same way, regardless of the input order.
- **Shell Sort:** The best case is around $\Omega(n \log n)$, while the average case is approximately $\Theta(n^{1.5})$ depending on the gap sequence. The worst case is $O(n^2)$, which can occur when the gap sequence does not effectively reduce disorder.
- **Radix Sort:** The time complexity is $\Omega(nk)$ in the best case, $\Theta(nk)$ on average, and $O(nk)$ in the worst case. It depends on the number of digits (k) and not on the input order, so all input types behave similarly.

Q2: Do the empirical results (running times of your algorithm implementations) match the theoretical asymptotic complexities? If there are discrepancies, explain the possible reasons.

Answer:

In general, the empirical results are consistent with the theoretical time complexities of the algorithms. For example, Insertion Sort performs very fast on sorted input and very slow on large random or reverse sorted inputs, which matches its $O(n^2)$ behavior. Similarly, Merge Sort shows stable performance across all input types, as expected from its $O(n \log n)$ complexity.

Quick Sort behaves efficiently on random input but performs much worse on sorted input. This is because the last element is used as the pivot, which causes the algorithm to reach its worst-case behavior $O(n^2)$.

However, some small differences can be observed between theoretical and experimental results, especially for smaller input sizes. These differences are mainly caused by system-related factors such as CPU caching, JVM optimizations, and measurement overhead. Additionally, constant factors and implementation details can also affect the actual running time.

Overall, the experimental results support the theoretical analysis, with minor deviations due to practical factors.

Q3: Compare the performances of Quick Sort and Merge Sort across different input orderings (random, sorted, reverse-sorted). Discuss how their algorithmic strategies influence performance consistency.

Answer:

Quick Sort and Merge Sort show different behaviors depending on the input order. Quick Sort performs very efficiently on random input, since the pivot usually splits the array into balanced parts, resulting in $O(n \log n)$ time. However, for sorted and reverse sorted inputs, its performance decreases significantly. This happens because the implementation selects the last element as the pivot, which leads to unbalanced partitions and causes the algorithm to reach its worst-case time complexity $O(n^2)$.

On the other hand, Merge Sort shows consistent performance across all input types. Its running time remains $O(n \log n)$ regardless of whether the input is random, sorted, or reverse sorted. This is because Merge Sort always divides the array into equal parts and merges them in a fixed manner, independent of input order.

In summary, Quick Sort is faster in practice for random data but less stable in terms of performance, while Merge Sort is more consistent and predictable due to its algorithmic structure.

Q4: Compare the performance of Shell Sort and Insertion Sort across different dataset sizes. At what input size does Shell Sort begin to significantly outperform Insertion Sort in your experiments, and why?

Answer:

From the experimental results, Insertion Sort performs well for very small input sizes, especially when the data is already sorted. However, as the input size increases, its running time grows rapidly due to its $O(n^2)$ complexity.

Shell Sort, on the other hand, performs better as the dataset size increases. In the experiments, Shell Sort starts to significantly outperform Insertion Sort around input sizes of 4000–8000 elements. After this point, the difference becomes more noticeable, especially for larger inputs.

The main reason for this difference is that Shell Sort reduces the number of required comparisons and swaps by using gap values. This allows elements to move faster to their correct positions compared to Insertion Sort, which only compares adjacent elements. As a result, Shell Sort scales better for larger datasets.

Overall, Insertion Sort is suitable for small inputs, while Shell Sort becomes more efficient for medium and large input sizes.

Q5: Given that Radix Sort is a non-comparison-based sorting algorithm, discuss how it handles large numerical ranges. Under what conditions does Radix Sort outperform comparison-based algorithms?

Answer:

Radix Sort processes numbers digit by digit instead of comparing them directly. Because of this, its performance depends on the number of digits rather than the magnitude of the values. Even if the numerical range is large, the algorithm can still perform efficiently as long as the number of digits is limited.

Radix Sort outperforms comparison-based algorithms when the number of digits (k) is small compared to the number of elements (n). In such cases, its time complexity $\Theta(nk)$ becomes close to linear, making it faster than algorithms like Quick Sort or Merge Sort, which have $O(n \log n)$ complexity.

However, if the number of digits becomes large, the performance of Radix Sort may decrease because it needs more passes over the data. Also, it requires additional memory for auxiliary arrays, which can be a disadvantage in some cases.

Overall, Radix Sort is most effective for large datasets with relatively small digit sizes, such as integer values with fixed length.